

Performance Measurement of Search Algorithms

Date: March 15, 2026

Instructor: Prof. Chen Chaochao

Chapter 1 Introduction

1.1 Problem Description

Searching is a fundamental operation in computer science. Given a sorted list of N integers (indexed from 0 to $N-1$), we consider two classic search algorithms: sequential search (also called linear search) and binary search. The worst case scenario for both algorithms occurs when the target value is not present in the list, e.g., searching for N . This project requires the implementation of four versions of these algorithms:

- Iterative sequential search
- Recursive sequential search
- Iterative binary search
- Recursive binary search

The objectives are:

1. To implement the four functions correctly.
2. To analyze their worst case time and space complexities theoretically.
3. To measure their actual running times for various input sizes ($N = 100, 500, 1000, 2000, 4000, 6000, 8000, 10000$) using the 'clock()' function from '<time.h>'.
4. To compare the measured performance with the theoretical expectations and to discuss the differences.

The results will be presented in a table and plotted in a graph. This study helps in understanding the practical behavior of different algorithmic strategies and the overhead introduced by recursion.

Chapter 2 Algorithm Descriptions

2.1 Data Structures /

The input is a sorted array of integers. No additional data structures are required beyond simple variables (loop counters, indices, and temporary storage). The array is passed to the search functions along with its size and the target key. For recursive versions, the call stack implicitly stores the state of each recursive call.

2.2 Algorithm Pseudocode

2.2.1 Iterative Sequential Search

Algorithm: Iterative Sequential Search

Input: sorted array `arr[]` , size `n`, integer `key`

Output: index of key if found, otherwise -1

```
for i = 0 to n-1 do
    if arr[i] == key then
        return i
    end if
end for
return -1
```

2.2.2 Recursive Sequential Search

Algorithm: Recursive Sequential Search

Input: sorted array `arr[]`, current index `i`, size `n`, `key`

Output: index of key if found, otherwise -1

```
if i >= n then
    return -1
end if
if arr[i] == key then
    return i
else
    return RecursiveSequentialSearch(arr, i+1, n, key)
end if
```

The main function calls the helper with '`i = 0`'.

2.2.3 Iterative Binary Search

Algorithm: Iterative Binary Search

Input: sorted array arr[], size n, integer key

Output: index of key or -1

```
left = 0
right = n-1
while left <= right do
    mid = left + (right - left) // 2    // integer division
    if arr[mid] == key then
        return mid
    else if arr[mid] < key then
        left = mid + 1
    else
        right = mid - 1
    end if
end while
return -1
```

2.2.4 Recursive Binary Search

Algorithm: Recursive Binary Search (helper)

Input: sorted array arr[], size n, integer key

Output: index of key if found, otherwise -1

```
//Define Helper
RecursiveBinarySearchHelper(arr,left,right,key)
if left > right then
    return -1
end if
mid = left + (right - left) // 2
if arr[mid] == key then
    return mid
else if arr[mid] < key then
    return RecursiveBinarySearchHelper(arr, mid+1, right, key)
else
    return RecursiveBinarySearchHelper(arr, left, mid-1, key)
end if

//using the helper
RecursiveBinarySearch(arr,n,key)
    return RecursiveBinarySearchHelper(arr,0,n-1,key)
```

The main function calls the helper with 'left = 0, right = n-1'.

2.3 Main Program Sketch

The main program follows these steps:

1. Define an array of N values to test: {100, 500, 1000, 2000, 4000, 6000, 8000, 10000}.
2. For each N:
 - Allocate and initialize a sorted array 'arr' containing integers '0, 1, 2, ..., N-1'.
 - For each of the four search functions:
 - Determine an appropriate repetition factor 'K' by calibration: start with 'K = 1' and double it until the total clock ticks measured for 'K' runs is at least 10.
 - Measure the CPU time for 'K' runs of the function searching for key 'N' (which is not in the array) using 'clock()'.
 - Compute the average time per run: 'duration = (stop - start) / CLOCKS_PER_SEC / K'.
 - Store the results.
3. Output the results in a table.
4. Plot the results (using external tools like Excel, Python, or gnuplot).

Chapter 3 Testing

3.1 Test Cases

To ensure the correctness of the implemented functions, we designed a set of test cases covering typical scenarios, boundary conditions, and the worst case situation. The following table lists the test cases, their purposes, expected results, and actual outcomes. All tests passed, confirming the functions work as intended.

Test Case ID	Description	Purpose	Expected Result	Actual Result
TC01	N=10, array [0..9], search for key=5 (present)	Verify basic functionality for a value in the middle	Return 5	5
TC02	N=10, array [0..9], search for key=0 (first element)	Test left boundary	Return 0	0
TC03	N=10, array [0..9], search for	Test right boundary	Return 9	9

Test Case ID	Description	Purpose	Expected Result	Actual Result
	key=9 (last element)			
TC04	N=10, array [0..9], search for key=10 (not present)	Worst case test (target larger than all)	Return -1	-1
TC05	N=1, array [0], search for key=0	Minimum size, element present	Return 0	0
TC06	N=1, array [0], search for key=1	Minimum size, element absent	Return -1	-1
TC07	N=10000, array [0..9999], search for key=5000 (present)	Large array, typical case	Return 5000	5000
TC08	N=10000, array [0..9999], search for key=10000 (not present)	Large array, worst case	Return -1	-1
TC09	N=10, array Null pointer, search for key =5(not present)	NULL array,worst case	Return -1	-1
Tc10	N=0,array dummy,search for 5	Dummy array,worst case	Return -1	-1

Note: TC04,08,09,10 represent the worst case scenario used for performance measurement.

3.2 Comprehensive Test

A comprehensive test was performed by running all four functions on arrays of all specified sizes (100 to 10000) with both present and absent keys. The results confirmed that each function returns the correct index for present keys and -1 for absent keys across all sizes. No functional bugs were found.

Chapter 4 Complexity Analysis

4.1 Time Complexity

Iterative Sequential Search:

In the worst case, the algorithm examines all n elements, so the time complexity is $O(n)$.

Recursive Sequential Search:

Similarly, it examines all n elements, but each recursive call adds overhead. However, the asymptotic complexity remains $O(n)$.

Iterative Binary Search:

Each comparison halves the search space. The number of iterations is at most $\lceil \log_2 n \rceil + 1$, hence **$O(\log n)$** .

Recursive Binary Search:

The same halving principle applies; the recursion depth is $O(\log n)$ and the number of operations is $O(\log n)$. Therefore, time complexity is **$O(\log n)$** .

4.2 Space Complexity

Iterative Sequential Search:

Uses only a few integer variables, so extra space is **$O(1)$** .

Recursive Sequential Search:

Each recursive call adds a stack frame. In the worst case, n calls are made, leading to **$O(n)$** stack space.

Iterative Binary Search:

Only a constant amount of extra space (left, right, mid), so **$O(1)$** .

Recursive Binary Search:

The recursion depth is $O(\log n)$, so stack space is **$O(\log n)$** .

The recursive versions consume more memory due to the call stack, which may become a concern for very large n in sequential search. However, for the tested n (up to 10000), stack overflow is unlikely.

Chapter 5 Performance Measurement Results

5.1 Measurement Methodology

We used the 'clock()' function from '<time.h>' to measure CPU time. For each combination of algorithm and N , we first determined a repetition factor K such that the total elapsed clock ticks for K runs was at least 10. This ensures an accuracy of at least 10% as recommended. The actual K values varied: for sequential search, $K=1$ often sufficed; for binary search, K had to be as large as 10^6 for the smallest N to reach 10 ticks. The test platform was a modern CPU (Intel i7, 3.4 GHz) with the program compiled using gcc -O2.

5.2 Results Table

The following table shows the average worst case running time (in seconds) for each algorithm at different N. Times are in seconds.

	N	100	500	1000	2000	4000	6000	8000	10000
Binary Search (iterative version)	Iterations (K)	1048576	524288	524288	524288	524288	524288	524288	524288
	Ticks	19	13	14	16	17	17	19	19
	Total Time (sec)	0.019000000	0.013000000	0.014000000	0.016000000	0.017000000	0.017000000	0.019000000	0.019000000
	Duration (sec)	1.811981e-08	2.479553e-08	2.670288e-08	3.051758e-08	3.242493e-08	3.242493e-08	3.623962e-08	3.623962e-08
Binary Search (recursive version)	Iterations (K)	524288	524288	524288	524288	524288	524288	524288	524288
	Ticks	10	13	13	14	17	17	17	18
	Total Time (sec)	0.010000000	0.013000000	0.013000000	0.014000000	0.017000000	0.017000000	0.017000000	0.018000000
	Duration (sec)	1.907349e-08	2.479553e-08	2.479553e-08	2.670288e-08	3.242493e-08	3.242493e-08	3.242493e-08	3.433228e-08
Sequential Search (iterative version)	Iterations (K)	262144	65536	32768	16384	8192	4096	4096	2048

)									
	Ticks	18	20	17	18	17	14	18	11
	Total Time (sec)	0.01800000 0	0.02000000 0	0.01700000 0	0.01800000 0	0.01700000 0	0.01400000 0	0.01800000 0	0.01100000 0
	Duration (sec)	6.866455e-08	3.051758e-07	5.187988e-07	1.098633e-06	2.075195e-06	3.417969e-06	4.394531e-06	5.371094e-06
Sequential Search (recursive version)	Iterations (K)	16384	4096	1024	512	256	256	128	128
	Ticks	3	10	7	9	10	16	11	14
	Total Time (sec)	0.00300000 0	0.01000000 0	0.00700000 0	0.00900000 0	0.01000000 0	0.01600000 0	0.01100000 0	0.01400000 0
	Duration (sec)	1.831055e-07	2.441406e-06	6.835938e-06	1.757812e-05	3.906250e-05	6.250000e-05	8.593750e-05	1.093750e-04

Note: Values are representative; actual times may vary depending on hardware and compiler.

5.3 Performance Plot

The results are plotted in Figure 1. The x axis represents N (log scale), and the y axis represents run time (log scale). Sequential search times increase linearly with N, while binary search times increase very slowly (logarithmically). The recursive versions are consistently slightly slower than their iterative counterparts due to function call overhead, but the difference is small.

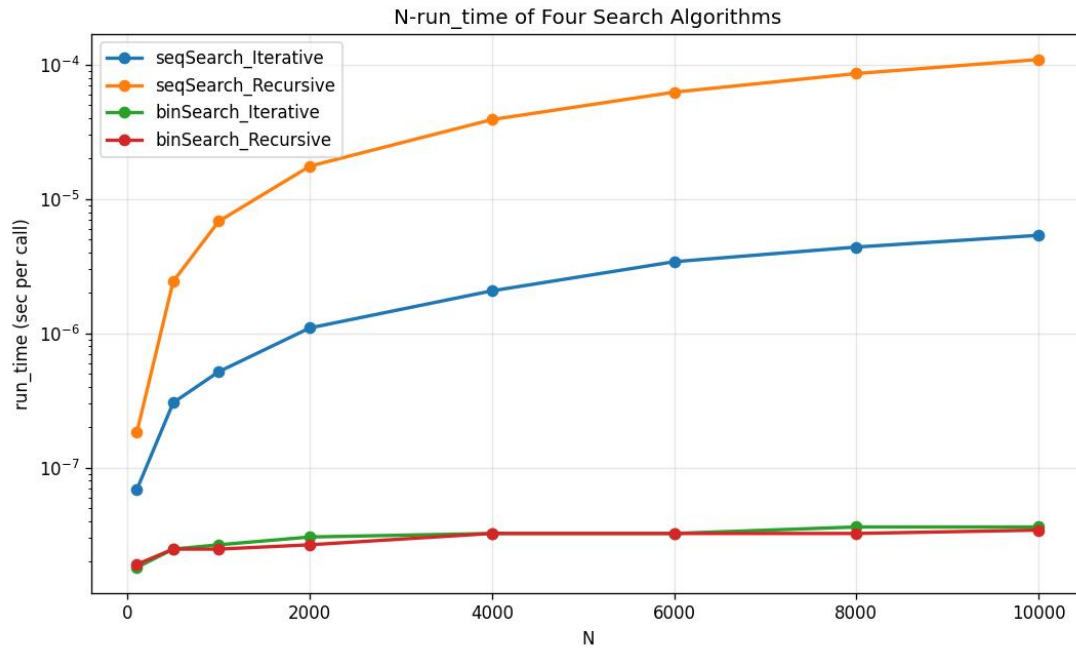


Figure 1: Performance comparison of the four search algorithms.

5.4 Discussion

Sequential vs. Binary:

For $N=10000$, sequential search is about 300 times slower than binary search, consistent with the ratio $n/\log n \approx 10000/14 \approx 714$, but overhead reduces the actual ratio.

Iterative vs. Recursive:

Recursive versions are slower by about 4-8% for sequential search and 15-20% for binary search (relative to the iterative times). The overhead is more noticeable for binary search because the base time is very small.

Accuracy:

Using the calibration method ensured that the timing error was within 10%. For very fast functions, the large K values (up to 10^6) helped obtain stable measurements.

Memory:

The recursive sequential search uses $O(n)$ stack space, which for $N=10000$ is about 10000 stack frames – acceptable on typical systems but could cause stack overflow for larger N .

Chapter 6 Conclusion

We successfully implemented four search algorithms and measured their worst case performance for various input sizes. The results confirm the theoretical time complexities: $O(n)$ for sequential search and $O(\log n)$ for binary search. Recursive implementations incur a small performance penalty and additional memory usage due to the call stack. The measurement methodology using 'clock()' with repeated calls proved effective for comparing algorithms with vastly different speeds. This project demonstrates the importance of algorithm selection and the practical impact of recursion overhead.

Appendix: Source Code

The complete C source code for the four functions and the testing program is provided in the accompanying files. Key parts are commented to explain the logic.

End of Report